AI ASSISTANT CODING

ASSIGNMENT - 10.1

Name: Aman Sarkar

Ht.no: 2303A51273

Batch: 05

Task Description #1 – Syntax and Logic Errors:

Prompt: The above syntax.py has some logical errors and syntax errors using ai correct and fix the errors.

```
the above syntax.py has some logiacl error
and syntax errors using ai correct and fix
logical errors
```

Code

```python
def calc_average(marks):
    total = 0
    for m in marks:
        total += m
    average = total / len(marks)
    return average

marks = [85, 90, 78, 92]
print("Average Score is", calc_average(marks))
```

Output:

```
C:\Users\thaku\OneDrive\Desktop\Training>C:/Users/thaku/AppD
ata/Local/Programs/Python/Python313/python.exe c:/Users/thak
u/OneDrive/Desktop/Training/syntax.py
Average Score is 86.25
```

Explanation:

Indentation Error

- Statements inside the function were not indented.
- Python requires proper indentation to define function blocks.

Variable Name Typo

- You wrote return avrage instead of return average.
- This causes a NameError because avrage is not defined.

Missing Parenthesis

- The print() statement was missing a closing).
- This causes a SyntaxError.

Block Structure Issue

- The for loop body was not indented properly.
- Python cannot identify which statements belong inside the loop.

Task Description #2 – PEP 8 Compliance
Prompt: for the given code use ai to refactor code to follow PEP style

```
def area_of_rect(L,B) : return L*B
print(area_of_rect(10,20))
use ai to refactor to folloe PEP 8 style
```

Code:

```
def area_of_rect(L, B):
    return L * B
print(area_of_rect(10, 20))
```

Output:

```
C:\Users\thaku\OneDrive\Desktop\Training>C:/Users/thaku/AppD
ata/Local/Programs/Python/Python313/python.exe c:/Users/thak
u/OneDrive/Desktop/Training/syntax.py
200
```

Explanation:

- Used descriptive function name (area_of_rectangle instead of area_of_rect)
- Used meaningful parameter names (length, breadth instead of L, B)
- Added proper spacing around operators (length * breadth)
- Removed inline function definition (function written in proper block format)
- Added a docstring for better documentation
- Added blank line after function definition (improves readability)

Task Description #3 – Readability Enhancement

Prompt: Improve the readability of the following Python code without changing its logic or output. Use meaningful variable and function names, proper indentation, and add simple comments.

```
def c(x,y):
return x*y/100
a=200
b=15
print(c(a,b))
Improve the readability of the following Python code without changing
its logic or output.
Use meaningful variable and function names, proper indentation, and
add simple comments.
```

Code:

```python
# Function to calculate percentage of an amount
def calculate_percentage(amount, percent):
    return amount * percent / 100

# Define the principal amount and percentage rate
principal_amount = 200
percentage_rate = 15

# Calculate and print the result
print(calculate_percentage(principal_amount, percentage_rate))
```

Output:

```
C:\Users\thaku\OneDrive\Desktop\Training>C:/Users/thaku/AppData/Local/Programs
/Python/Python313/python.exe c:/Users/thaku/OneDrive/Desktop/Training/syntax.p
y
30.0
```

Explanation:

Renamed c → calculate_percentage and parameters/vars to describe their roles.

- Added a docstring and comments for clarity.
- Fixed indentation and spacing so the code is easy to read.
- Logic unchanged; it still prints 30.0 for the given inputs.

Task Description #4 – Refactoring for Maintainability

Prompt: Refactor the following Python code to improve maintainability. Break repetitive or long code into reusable functions without changing the output**.**

```
Refactor the following Python code to improve
maintainability.
Break repetitive or long code into reusable functions without
changing the output.
```

Code:

```python
def welcome_student(name):
    print("Welcome", name)
students = ["Alice", "Bob", "Charlie"]
for student in students:
    welcome_student(student)
```

Output:

```
C:\Users\thaku\OneDrive\Desktop\Training>C:/Users/thaku/AppData/Local/Prog
rams/Python/Python313/python.exe c:/Users/thaku/OneDrive/Desktop/Training/
syntax.py
Welcome Alice
Welcome Bob
Welcome Charlie
```

Explanation:

- Pulled the welcome message into a greet function.
- Created welcome_all to loop through any student list.
- Data (names) now lives in the list, not hard coded print calls.
- Output stays the same but structure is easier to extend.

Task Description #5 – Performance Optimization

Prompt: Optimize the above code using list comprehensions or vectorized operations.

```
optimize the above code using list comprehensions or
vectorized
operations.
```

Code:

```
nums = [i for i in range(1, 1000000)]
squares = [n**2 for n in nums]
print(len(squares))
```

Output:

```
C:\Users\thaku\OneDrive\Desktop\Training>C:/Users/thaku/AppData/Local/Prog
rams/Python/Python313/python.exe c:/Users/thaku/OneDrive/Desktop/Training/
syntax.py
999999
```

Explanation:

Used range() instead of creating a list

- range(1, 1000000) generates numbers only when needed.
- This reduces memory usage compared to storing all numbers in a list.

Replaced for loop and append() with list comprehension

- List comprehensions execute faster than traditional loops.
- They reduce overhead caused by repeated function calls.

Simplified the calculation

- Used n * n instead of n**2.
- Multiplication is faster than exponentiation.

Reduced code complexity

- Fewer lines of code make the program cleaner and easier to read.
- Improves maintainability and execution speed.

Improved overall performance

- The optimized code runs faster and uses less memory.
- Suitable for handling large datasets efficiently.

Task Description #6 – Complexity Reduction

Prompt: Identify the error and fix the error with elseif

identify the error and fix the error with elseif

Code:

```
def grade(score):
    if score >= 90:
        return "A"
    elif score >= 80:
        return "B"
    elif score >= 70:
        return "C"
    elif score >= 60:
        return "D"
    else:
        return "F"
```

Output:

```
pData/Local/Programs/Python/Python313/python.exe c:/Users/
thaku/OneDrive/Desktop/Training/syntax.py
Score: 90, Grade: A
```

Explanation:

Replaced nested if-else blocks with elif

- Reduces unnecessary nesting.
- Makes the logic easier to understand.

Improved code readability

- The grading conditions are now clearly ordered.
- Each condition is checked only when the previous one fails.

Reduced logical complexity

- Eliminates deep indentation levels.
- Easier to debug and maintain.

Same functionality with fewer lines

- Output remains unchanged.
- Code is more concise and clean.

Better performance and maintainability

- Less branching improves logical flow.
- Suitable for future updates or modifications.